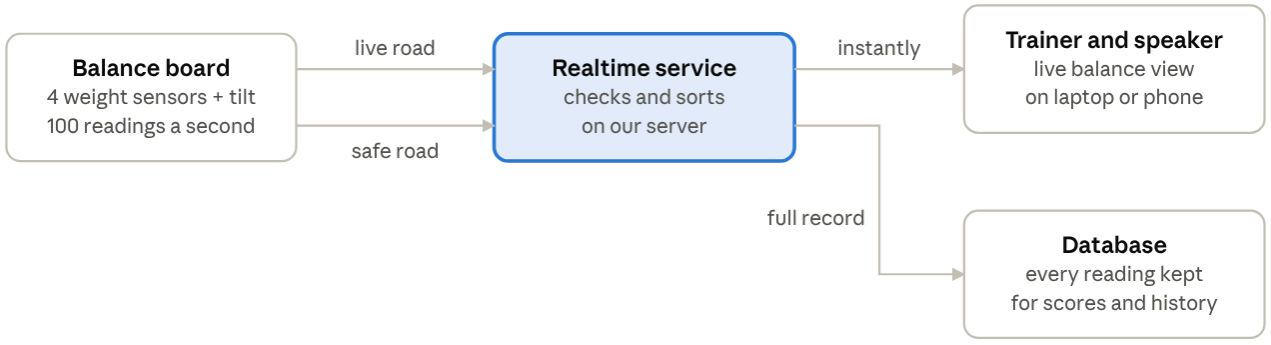
# How PIVOT's live balance data works: a plain-language guide

Nov 11, 2026 · @kokos

## The big picture

When a speaker stands on the PIVOT board, it measures their balance 100 times a second. Those measurements travel over Wi-Fi and the internet to our server. The server passes them straight on to the trainer's and speaker's screens, and also saves a complete copy for later.

**Each reading takes a fast road to the screens and a safe road to the archive**




the journey of one balance reading

In short, step by step:

1. **The board measures.** Four weight sensors under the corners and a tilt sensor record how the speaker leans and shifts.
2. **The board sends.** Each measurement goes out twice: once immediately for the live view, and again later in a bundle for the permanent record.
3. **The server sorts.** It checks the data really comes from a registered board and works out which training session it belongs to.
4. **The screens show it.** The live data is forwarded within a fraction of a second, so the speaker can correct their posture while still talking.
5. **The database keeps it.** The full record is stored, so each talk can get a stability score and be reviewed afterwards.

## The main parts

The system has five parts. Each has one job, so a problem in one part is easier to find and fix.

| Part | What it is | Everyday comparison |
|---|---|---|
| **Board** (ESP32 chip) | A small computer inside the balance board that reads the sensors and talks to Wi-Fi | A reporter in the field, phoning in what they see |
| **Realtime service** | A program on our server that receives board data and pushes it to screens instantly | A switchboard operator connecting calls |
| **Web app** (Django) | The website and app back end: logins, sessions, history pages | The front desk that handles bookings and paperwork |
| **Database** (Postgres and TimescaleDB) | Where everything is stored: users, boards, sessions, and every balance reading | The archive room, with a special shelf for measurements in time order |
| **Screens** (browser and mobile app) | What the trainer and speaker look at during and after a talk | The TV showing the live broadcast |

The board and the realtime service handle the fast, live work. The web app and database handle the slower, careful work, like accounts and history. Keeping the two sides apart means a busy website can never slow down the live feedback.

## Two roads for data

The board sends every measurement along two roads, because no single road is both instant and guaranteed. One road is fast but may lose a little; the other is slower but loses nothing.

| | Fast live road (UDP) | Safe complete road (TCP) |
|---|---|---|
| Comparison | A live radio broadcast | Registered mail with a signed receipt |
| When it is sent | Each reading, the moment it is taken (every 1/100 of a second) | Readings bundled together every few seconds |

| | Fast live road (UDP) | Safe complete road (TCP) |
|---|---|---|
| If something gets lost | It is skipped; the next reading arrives a moment later anyway | The board sends it again until the server confirms it arrived |
| If Wi-Fi drops | Live view pauses | The board keeps the readings in memory and sends them once Wi-Fi is back |
| Used for | The live balance view on screen | The permanent record and the stability score |

**Why skipping is fine for the live view.** On the live screen, only the newest reading matters. A reading from half a second ago is already out of date, so waiting for it to be resent would only make the screen lag.

**Why nothing may be lost in the record.** The stability score and the history charts need the whole talk. So the safe road keeps a numbered list. Every reading has a number, and the server confirms which numbers it has received. Missing numbers are resent, and numbers that arrive twice are only stored once.

**Keeping messages small.** Each reading is sent in a compact, fixed layout of 56 bytes, not as readable text. That is roughly the size of one short text message. A small, fixed size means the board can hold about six times more readings in its memory while Wi-Fi is down than it could as text.

## Getting data onto the screen

The screens get live data through **WebRTC**, the same technology video-call apps use to send voice and video with almost no delay. We don't send video. We use WebRTC's *data channels*, which are like private live lines between the server and each screen.

Each screen gets two lines, set up differently on purpose:

| Line | Behaves like | Carries |
|---|---|---|
| **Stability line** | A live sports ticker: if one update is missed, the next one replaces it | Balance readings, about 100 per second |
| **Events line** | A chat conversation: every message arrives, in order | The live transcript, invitations, analysis results, warnings |

The live speech transcript already uses this setup, so the balance data simply joins it as a new type of message. The screen redraws its balance picture in step with the display's own refresh, not once per message. This keeps the animation smooth and stops the phone

or laptop from wasting effort.

The server is also kept free of slow jobs. Turning speech into text (Whisper) needs a lot of computing power, so it runs separately. A long transcription can't hold up the balance readings.

## Knowing whose data is whose

The board doesn't know who is standing on it. It only knows its own ID number. The server keeps track of who uses which board and when, a bit like a cloakroom matching coat tickets to coats.

1. **The trainer starts a session** in the app and picks a board, for example Board 7. A board can only be in one running session at a time.
2. **The server tells Board 7 to start streaming.** It also notes: "Board 7 now belongs to this session".
3. **Readings arrive with only the board's ID.** The server looks up its note and sends them to that session's screens.
4. **Talks are matched by time.** A session can contain several talks. Each talk has a start and end time, so every reading belongs to the talk that was running when it was measured.

Matching by the time a reading was *measured*, not the time it *arrived*, matters after a Wi-Fi drop. Readings resent a minute late still land in the right talk.

For time matching to work, the board's clock and the server's clock must agree. The board sets its clock from an internet time service, and the server then regularly checks how far apart the two clocks are and corrects for the difference. It works much like two people saying "it's 10:00 now" over the phone and allowing for the call delay.

## Keeping data safe and private

The server is reachable from the internet, so anyone could try to send it fake data. Four rules protect the system and its users:

- **Every board has a secret key.** It is given to the board once during setup over Bluetooth, from the trainer's phone, and is never sent over the internet afterwards.
- **Every live message carries a signature.** The board uses its secret key to stamp each message, like a wax seal. The server checks the seal, and messages with a missing or wrong seal are thrown away.
- **The complete record travels encrypted.** The safe road uses TLS, the same protection as the padlock in a browser's address bar, so nobody in between can read or change it.

- **Balance data carries no names.** Readings are stored with ID numbers for the board, session and talk only. Names and emails stay in the user account, so balance data can be anonymised or deleted together with an account (GDPR).

## Why speed is hard

The team's goal is for a movement on the board to appear on screen within **50 milliseconds**, a twentieth of a second. At that speed, the feedback feels instant and helps the speaker rather than distracting them.

The tricky part is that the data takes a detour. The board and the screen are usually in the same room, but the data goes up to a server in a data centre and back down again, a bit like sending a letter to the person next to you through the post office. Every leg of that trip uses up part of the 50 ms:

| Leg of the trip | Rough time |
|---|---|
| Waiting for the next reading | up to 10 ms |
| Board to the Wi-Fi router | a few ms |
| Across the internet to the server | 10–30 ms |
| Server passes it on | under 1 ms |
| Server back across the internet to the screen | 10–30 ms |
| Screen waits to redraw | up to 17 ms |

These are estimates, so the team will measure the real times during testing. Three design choices keep the trip short:

- **Wi-Fi power saving is switched off on the board.** By default the chip naps between messages to save battery, and that alone could add up to about 0.3 seconds.
- **Each reading is sent on its own**, the moment it is taken, instead of waiting to fill a bundle.
- **Live data never touches the database on the way.** The server forwards it straight from the board to the screens.

If measurements show the internet legs alone take too long, the report will keep 50 ms as the goal and 250 ms as the firm limit, which the original requirement allows.

# Glossary

| Term | Plain meaning |
|---|---|
| aiortc | The software library that lets our server speak WebRTC |
| Bluetooth (BLE) | Short-range wireless, used only once to set up a new board from a phone |
| Database | Organised long-term storage for all the system's data |
| Django | The framework the website and app back end are built with |
| ESP32 | The small, cheap computer chip with built-in Wi-Fi inside the board |
| GDPR | The EU privacy law that sets how personal data must be handled |
| Latency | Delay: the time between a movement and seeing it on screen |
| Millisecond (ms) | One thousandth of a second |
| Packet | One small message sent over a network |
| Server | A computer in a data centre that runs our system around the clock |
| Session | One training meeting on a board, which can contain several talks |
| Signature (HMAC) | A stamp made with a secret key, proving a message is genuine and unchanged |
| TCP | The network method that confirms delivery and resends anything lost |
| TimescaleDB | An add-on to the database designed for measurements stored in time order |
| TLS | Encryption that keeps data private on the way, as in "https" |
| UDP | The network method that sends right away without checking delivery: fast, but may drop a message |

| Term | Plain meaning |
| --- | --- |
| VPS | A rented server in a data centre |
| WebRTC | Real-time technology from video calls, used here to push live data to screens |